Unité 8 — Cahier de laboratoire (étudiant)

Livrable C — Commandes, chemins & XSS · UE « Software Security »

Code UE 7532 14 U32 D1 · Document étudiant · 11 octobre 2026

Cadre. Application volontairement vulnérable, en local uniquement (livrable E).

Lab 8.1 — Injection de commande & traversée de chemin EXERCICE GUIDÉ

1. Objectifs

- Exécuter une commande arbitraire via une entrée non assainie.
- Lire un fichier hors du dossier prévu.

2. Prérequis

- Cours U8 §1–2.

3. Environnement

Image (E) + Flask, application fournie (U7).

4. Fichiers

APP.PY — /PING

```
@app.route("/ping")
def ping():
    host = request.args.get("host","127.0.0.1")
    out = subprocess.check_output("echo pong: " + host, shell=True)  # VULN : shell=True
    return out
```

APP.PY — /FILE

```
name = request.args.get("name","notes.txt")
with open(os.path.join("/.../docs", name)) as f:         # VULN : pas de confinement
    return f.read()
```

5. Commandes

SHELL

```
FLASK_APP=app.py flask run --port 5001
curl "http://127.0.0.1:5001/ping?host=127.0.0.1;id"
curl "http://127.0.0.1:5001/file?name=../../../../../etc/passwd"
```

6. Manipulations

a. Appelez `/ping` avec `host=127.0.0.1;id` : la sortie contient le résultat de `id`.

b. Appelez `/file` avec une suite de `../` menant à `/etc/passwd` : ajustez la profondeur.

7. Résultats attendus

`/ping` affiche `uid=0(root)…` (commande injectée exécutée) ; `/file` renvoie le contenu de `/etc/passwd`.

8. Explications

`shell=True` interprète `;id` ; l'absence de confinement laisse `../` remonter l'arborescence.

9. Questions

1. Quelle commande a été exécutée via `/ping` ?
2. Combien de `../` faut-il, et pourquoi ?
3. Quelles remédiations pour chacune ?

11. Vulnérabilité & remédiation

CWE-78 (OS command injection), CWE-22 (path traversal). Remédiation : liste d'arguments sans shell + validation ; confinement `realpath` sous le dossier de base.

12. Durée

1 h.

10. Corrigé : livrable D, Lab 8.1.

Lab 8.2 — XSS réfléchi & correctifs EXERCICE GUIDÉ

1. Objectifs

- Déclencher un XSS réfléchi, puis valider les correctifs des trois failles.

2. Prérequis

- Lab 8.1.

3. Environnement

Image (E) + Flask, application fournie (U7).

4. Fichiers

APP.PY — /SEARCH

```
return "<h1>Resultats pour %s</h1>" % q          # VULN : sortie non encodee
```

APP_FIXED.PY — /SEARCH

```
return "<h1>Resultats pour %s</h1>" % escape(q)    # encodage de sortie
```

5. Commandes

SHELL

```
curl "http://127.0.0.1:5001/search?q=<script>alert(1)</script>"   # vulnerable
# version corrigee :
FLASK_APP=app_fixed.py flask run --port 5003
curl "http://127.0.0.1:5003/ping?host=127.0.0.1;id"
```

```
curl "http://127.0.0.1:5003/file?name=../../../../../etc/passwd"
curl "http://127.0.0.1:5003/search?q=<script>alert(1)</script>"
```

6. Manipulations

a. Injectez un `<script>` dans `/search` : observez qu'il est renvoyé tel quel.

b. Lancez la version corrigée et rejouez les trois attaques (cmd, path, XSS) : toutes doivent échouer.

7. Résultats attendus

Vulnérable : le script apparaît non encodé. Corrigé : 400 (hôte invalide), 403 (accès refusé), et `<script>` encodé.

8. Explications

L'encodage de sortie transforme `<` en `<` : le navigateur affiche le texte au lieu d'exécuter le script.

9. Questions

1. Quelle différence entre XSS réfléchi, stocké et DOM ?
2. Pourquoi l'encodage dépend-il du contexte (HTML vs attribut vs JS) ?
3. Quelles défenses complémentaires (CSP, HttpOnly) ?

11. Vulnérabilité & remédiation

CWE-79 (XSS). Remédiation : encodage de sortie contextuel ; CSP ; cookies HttpOnly.

12. Durée

1 h.

10. Corrigé : livrable D, Lab 8.2.

Exemples issus de meltypot ; application de labo auteur ; sorties vérifiées.